

# Agent Flow Reference

**NVIDIA · NXP · ASU**

Function-level call flow for all three agents

**green = our code · orange = organizer-provided**

Harikrishnan KC · Team Chip Convergence

# The shape all three share

- ①

DIAGNOSE

②

SELECT

③

MODEL

④

VERIFY

⑤

DECIDE
- zero tokens – read the real artifact, decide what to try

structure + risk + playbook → which transform to ask for

the ONLY probabilistic step – bounded, optional

deterministic gates – the model has no vote here

ship the best PROVEN result, else ship the baseline

|        | proof technology                | why                                                         |
|--------|---------------------------------|-------------------------------------------------------------|
| NVIDIA | formal equivalence (yosys LEC)  | a pristine reference exists — we can <i>prove</i> sameness  |
| NXP    | contracts + independent oracles | nothing to compare against — we're generating something new |
| ASU    | the organizer's own scorer      | the objective is directly measurable                        |

Same architecture, different guarantees — because each problem admits a different kind of proof.

# NVIDIA — phases 0–2 (no tokens spent yet)

```
nvidia_agent.py → ppa/controller.py :: run(ip, rounds, k, model, ...)
```

— phase 0 · setup + refusal —

```
_campaign_gate(spec, model, profile)      refuse if contract PENDING
                                           → NVDLA blocked HERE, before any call
```

```
evaluate.baseline(ip)
  ↳ evaluate.measure(ws, label)           harness/measure.sh → yosys + OpenSTA
    ↳ harness/_metrics.sh                 fail-closed metric extraction
_sdc_period(ip)                           clock period from the IP's own SDC
```

— phase 1 · diagnosis (OURS, zero tokens) —

```
diagnose.diagnose(ip) → DiagnoseResult(wns_ps, critical_files, structure)
  ↳ _run_flatten / _parse_timing           worst paths from the STA report
  ↳ _attribute_instance                    path cell → SOURCE FILE
  ↳ _parse_area                           area ranking fallback
```

```
sta_feedback.classify(cells) → TAG
  ↳ _family(cell_type)                    cell → xor / carry / mux / aoi / gate
    xor ≥ .35 & carry < .05 → xor-linear-network    (GF(2) crypto layer)
    xor+carry ≥ .20         → arith-carry-chain     (real adders)
    mux ≥ .25                → mux-select
    aoi ≥ .50                → control-boolean-network
    gate ≥ .35               → wide-gate-decode
```

— phase 2 · choose what to ask for (OURS) —

```
_TAG_TO_RUNGS[tag]           structure → candidate rungs
_risk_gated(preferred, setup, period)  p = max(0, -WNS)/period bands them
proposer.pick_strategies(k, weights, ...) k DISTINCT rungs (diversity ≠ temperature)
skills.retrieve([tag], k=5)         ranked -(helpful - 2×harmful), +3 'avoid'
```

## NVIDIA — phases 3–4 (the model, then the gates)

```
— phase 3 · the model (the only probabilistic step) —————
proposer.PromptContext(...) / build_prompt(ctx, strategy)
model.generate(prompt)           ← THE MODEL CALL
proposer.parse_response(ip, text) → {file: content}
proposer.fence_violation(ip, files) veto forbidden zones
                                   aes S-box · kmac DOM core · nvdla reset/CDC
```

```
— phase 4 · five-layer verification —————
evaluate.evaluate_many(cands, workers)
  └ evaluate.evaluate_one(cand, base)
      └ workspace.Workspace.create() / ws.overlay(files)
          └ _regen_sv2v(touched)           dual-representation IPs (.sv → .v)
      └ verify.lint                         1
      └ verify.compile_gate                 2
      └ verify.tb_gate                      3 fail-closed: nonzero rc = FAIL
      └ verify.lec(...) / run_lec_command 4 yosys PROOF
          PROVEN ⇔ rc==0 ∧ success ∧ total>0 ∧ proven==total ∧ unproven==0
      └ verify.dualsim(...)                 5 cycle-exact co-simulation
      └ evaluate.measure(ws)                area / WNS / cells / power
```

Anything short of all five conditions on layer 4 is **INCONCLUSIVE** — never a pass.

## NVIDIA — phases 5–6 (accept, learn, ship)

```
— phase 5 · accept / refuse —————
netlist-collapse guard          cells < 0.5×parent ⇒ measurement garbage
proposer.cdc_ff_violation(spec, base, ppa) multi-clock: FF count may NOT drop
objective.Objective.better(cand, parent)
objective.Objective.adp_ratio(cand, base) area × delay_ps, normalized
pool.DesignPool.add(...) / ParetoFrontier.offer(...)

— phase 6 · learn, then decide what ships —————
proposer.build_reflect_prompt / parse_reflection
skills.vote(id, helpful) / skills.add_bullet(...)      playbook evolves
plateau controller                                     stop when frontier stalls

_canonical_best(ip, pool, obj, base_ppa)
    searches the FULL POOL (not the frontier – a broken candidate can
    DOMINATE a proven one and evict it: the ascon lesson)
    keeps ONLY lec == "PROVEN"; re-checks cdc_ff_violation on persisted state
    → None ⇒ caller ships the BASELINE
_experimental_best(...)      the demoted unproven one, labelled
emit.staged_replace(...)     atomic emit + manifest, with rollback
```

**classify() decides what to try. \_canonical\_best() decides what ships.**

A heuristic in the search; a proof in the verdict.

## NXP — phases 0–2

```
nxp_agent.py → run(model, P, deep=False)
  P = Paths.dev() | Paths.from_info(info) ← runner contract
  model = StubModel | VertexModel | EndpointModel (runner always Endpoint)

— phase 0 · read the problem, derive the contract (OURS) —————
read_spec(P)                                architecture.html → text
P.tb_skeleton.read_text()                   the organizer's port contract
validators.skeleton_top_name(skel) → DUT NAME
  easy → secure_periph_soc · medium → noc_aes_soc · hard → crypto_soc
  never hardcoded – an unseen testcase may use any name
required_params_table / demo_exemplars / doc_irq_map

— phase 1 · specs: model proposes, validator disposes —————
prompt_yaml(spec, skel, req_tbl, exemplars)
model.generate(...) ← MODEL CALL 1
extract_blocks(resp, "yaml|yml")
validators.yaml_validator(yamls)            typed errors BEFORE any RTL exists
  ↳ validators.parse_flat_yaml(y)
  ↳ errors → re-prompt with them appended

— phase 2 · generate the IPs (deterministic) —————
generate_ip(y, i, P) → rtl_gen_lib/rtl_gen_main.py ← ORGANIZER'S library
  ↳ _rtl_gen_env() PyYAML present? else our fail-closed shim
  ↳ validators.patch_library_rtl(text) known contest-library bug fixes
  ↳ generator rejections → re-prompt the specs
```

## NXP — phase 3: the correctness firewall

```
for attempt in range(3):
    prompt_top(spec, skel, gen, top_name=top_name, problem=P.problem)
    {
        _caps(problem) / _clip(text, cap, what)    tier-aware budgets, LOUD truncation
        module_interfaces(gen_files)               authoritative port headers
    }
    model.generate(...)                            ← MODEL CALL 2 (+ ≤2 repair calls)
    extract_blocks(top_resp, "verilog|v|...")

    — four independent gates — ALL must pass —
    validators.port_contract(top, skel, top=top_name)
    {
        skeleton_contract(skel, top)               what the TB demands
        top_ports(text, top)                       what we declared (skips #(params)!)
    }
    validators.reset_lint(top, strict_names=(problem=="easy"))
    validators.structural_diff(top, gen_mods, top=top_name)
    {
        parse_instances / _assign_map              census · IRQ reach · bus connectivity
    }
    validators.instance_port_directions(top, gen)
    {
        _is_literal(expr)                         an OUTPUT may not be driven by a constant
    }

    clean → write {top_name}.v      |      violations → re-prompt with typed errors

    — after 3 attempts: mechanical last resort —
    validators.repair_constant_driven_outputs(top, gen)
        .m_rready(1'b1) → .m_rready()           legal Verilog; design ELABORATES
```

## NXP — phases 4–5 (gate + independent oracles)

```
— phase 4 · the gate —————
run_gate(P)
  tb = SELF_CHECK_TB    if problem == "easy"    ← our self-checking TB
    = P.tb_skeleton    otherwise                ← organizer's skeleton
  iverilog -g2005 -o sim *.v tb
  easy      → vvp sim → fractional score + first-fail STAGE localization
  non-easy  → "skeleton elaboration OK" (no self-check TB ships for these tiers)

— phase 5 · independent oracles (easy tier) —————
kat_engine.run_kat(out_dir, work)
  { gen_smoke / run_vectors / compare_lines / _load_calibration
  { KAT(golden) 79/79 vs ref_models.py    ← INDEPENDENT oracle: is it RIGHT?
  { KAT(model)  79/79 vs the model's own  ← is it SELF-CONSISTENT?
stg_diff (--deep)      dual-SoC differential, cycle-identical 3,662 cycles
ip_models.py           20 bit-accurate IP models (SyncFifo, AsyncFifo, SramSp/Dp,
                      CdcSync, AxiLiteSram, TlRouter, Aes128, PerfCounter ...)
```

**yaml\_validator()** rejects a bad spec before RTL exists. **instance\_port\_directions()** rejects a bad stitch before compile exists.

Catch it at the earliest layer where it is cheap and the error is precise.



## ASU — the SHIPPED agent (zero model calls)

```
official_submission/agent.py
python3 agent.py <info.json> --model <name>  ← --model ACCEPTED and IGNORED

main(argv)
├ json.loads(info_json)           paths from the runner
├ read layout_script             the original pya builder
├ _looks_like_pya_layout(text)  SAFETY: not a pya builder?
├                               → emit the ORIGINAL, untouched
├ repair(text)
├   ├── _strip_write(text)       remove the original's write() call
├   ├── via_bar_snippet()        append the KLayout repair snippet
├   │   └ _asu_bar_pair(...)     V2/M3, V4/M5, V5/M6 – the transform
├ write output_path
└ _log(...)                     usage write best-effort (read-only FS)
```

**Seven functions. No model. No network. Python stdlib only.**

Runs in `python:3.10-slim`, `--read-only`, **no KLayout binary**.

The repair is *deferred*: `via_bar_snippet()` emits KLayout code that runs later, inside the organizer's evaluator, where `pya` actually exists.

# ASU — the DEV agent (where the model lives)

```
agent/asu_agent.py :: run(P, model=None, max_calls=6)

— phase 1 · diagnose (zero tokens) —————
drc_digest.load(drc_report) → Digest
  | _classify(violation)      → RuleFinding per violation class
  | rule_library()           our DRC repair-rule library (drc_rules.json)
  | match_rule(finding)      violation class → candidate transform

— phase 2 · deterministic repair passes —————
repairs.via_bar_pass(text)      THE WINNER – array → one continuous bar
  | _asu_bar_pair(...)         the geometric rewrite
  | _asu_snap_coord / _asu_snap_layer  grid + layer discipline
repairs.grid_snap_pass(text)    evaluated, NOT shipped – couples badly

— phase 3 · model repair passes (dev only) —————
model_repair.make_model / build_prompt(...)  _rules_block() injects the rules
model_repair.propose_fix_pass(...)           best-of-N + render-error repair
  | _extract_code(resp)  | _compiles(code)    cheap reject before measuring
  | any model/endpoint failure NEVER aborts the run

— phase 4 · measure with the REAL scorer, keep best —————
verify.VContext / _load_evaluator()          loads the ORGANIZER'S evaluator
verify.measure(candidate, ctx) → Result
  | render with pinned KLayout 0.30.1        the version they score with
  | run the ASAP7 DRC deck                    → final_violation_rate
  | verify.connectivity_credible(...)         eligibility gate #2
    | _conn_signature(...)

keep-best across all passes → emit the winner
```

# ASU — why there are two agents

`verify.measure()` is the engine of the dev agent — and it cannot exist in the official container.

It needs KLayout at *agent runtime*. The official image has none and is read-only.  
So a measure-and-keep-best loop — deterministic **or** model-driven — is structurally impossible there.

Two options:

1. Ship an agent that measures nothing and falls back to emitting the original
2. **Extract the one transform that measurement had already proven, and ship it deterministically**

We did the second. The shipped agent's output is **byte-identical** to what the dev agent's keep-best loop selected — verified by SHA-256:

`fdae65dd` `8c21ce2f` `bacc0531` `a31d59ad` `e5851416`

**`verify.measure()` is our scorer BEING their scorer** — same pinned KLayout 0.30.1, same ASAP7 deck, plus the connectivity check mirroring eligibility gate #2.

**`_looks_like_pya_layout()` is the fail-safe** — on an unrecognized input we emit the original untouched. It fails toward doing nothing.

# Side by side

|          | NVIDIA                                                                               | NXP                                                       | ASU                                                      |
|----------|--------------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------|
| diagnose | <code>diagnose.diagnose()</code> <code>sta_feedback.classify()</code>                | <code>read_spec()</code> <code>skeleton_top_name()</code> | <code>drc_digest.load()</code> <code>match_rule()</code> |
| select   | <code>_TAG_TO_RUNGS</code> <code>_risk_gated()</code> <code>pick_strategies()</code> | <code>prompt_yaml()</code> <code>prompt_top()</code>      | <code>repairs.via_bar_pass()</code>                      |
| model    | 1 call per rung, k rungs/round                                                       | 2 calls (+≤2 repairs)                                     | none in the shipped agent                                |
| verify   | 5 layers, <code>verify.lec()</code> = proof                                          | 4 validators + KAT ×2 oracles                             | <code>verify.measure()</code> = the real scorer          |
| decide   | <code>_canonical_best()</code> — LEC-PROVEN only                                     | <code>run_gate()</code> + eligibility                     | keep-best by measured FVR                                |
| fallback | ship the <b>baseline</b>                                                             | ship best-effort RTL that <b>elaborates</b>               | emit the <b>original</b> untouched                       |

Every one of them has a defined **do-nothing** outcome that is safe.  
That is what makes an unseen input survivable.

## What is ours

Everything in `nvidia_work/agent/ppa/` , `nxp_work/agent/` , `asu_work/agent/` **and** `asu_work/official_submission/`

The organizer provides the **problem**: RTL sources, `rtl_gen_lib` , the DRC deck and evaluator, ASAP7 techlib, Dockerfiles, testbench skeletons.

There is no optimization tooling, no STA analysis, no structure classifier, no correctness firewall and no repair library in any contest repo — their Python is vendor utilities and test harnesses.